STA 3402 – Machine Learning & Neural Networks

Lecture 4 – Cluster Analysis: k-means, Hierarchical (Agglomerative සහ Divisive) Clustering

කථිකාචාර්ය: Dr. Sameera Viswakula, සංඛ්‍යාන අධ්‍යයන අංශය, කොළඹ විශ්වවිද්‍යාලය

Lecture recording එක, "Clustering Algorithms" slides සහ R code එක භාවිතා කර සකස් කළ සම්පූර්ණ සටහන් (සරල සිංහලෙන්).

මේ සටහන් වල ඇති සියලුම outputs සහ plots R code එක ඇත්තටම run කරලා ලබාගත්ත ඒවා.

මේ සටහන කියවන විදිහ

තාක්ෂණික වචන (technical terms) exam එකේදී සහ R code එකේදී ඉංග්‍රීසියෙන්ම එන නිසා ඒවා ඉංග්‍රීසියෙන්ම තියලා තේරුම සිංහලෙන් දීලා තියෙනවා. රූපවල සහ R output වල labels ඉංග්‍රීසියෙන් තියෙන්නේ ඒ නිසා.

පටුන

1. Clustering කියන්නේ මොකක්ද, ඇයි භාවිතා කරන්නේ?
2. පළමු practical එක: iris data මත k-means
3. Within සහ between sum of squares – clustering එකක් කොච්චර හොඳද?
4. සමස්ත චිත්‍රය: clustering වර්ග
5. කිරි උදාහරණය (slides)
6. දුර (Distance): Euclidean සහ Manhattan
7. Hierarchical clustering: පියවර සහ linkage ක්‍රම
8. Genes පහක් සමඟ single linkage උදාහරණය (g1–g5)
9. උදාහරණ 2: normal populations තුනකින් data
10. k-means clustering – න්‍යාය
11. උදාහරණ 3: simulate කළ data මත k-means සහ dendrograms
12. DIANA සමඟ divisive clustering
13. ගෙදර වැඩ (05.01.2026 ට කලින්)
14. අමතර පුහුණුව: මාළු data සහ penguins data
15. සාරාංශය

1. Clustering කියන්නේ මොකක්ද, ඇයි භාවිතා කරන්නේ?

> **අර්ථ දැක්වීම (slides)**
> **Clustering** කියන්නේ objects කිහිපයක් කණ්ඩායම් (groups) හෝ **clusters** ගණනකට කාණ්ඩ කිරීමේ ක්‍රියාවලිය.
> **අරමුණ:** එකම cluster එකේ ඇති objects, වෙනත් clusters වල ඇති objects වලට වඩා **එකිනෙකට සමානයි**.

- Variable 1 සහ variable 2 හි scatter plot එකක් හිතන්න. සමහරවිට **කණ්ඩායම්** පේන්න පුළුවන්.
- මුළු data set එකටම **එක** regression model එකක් fit කරනවා වෙනුවට **හැම cluster එකකටම වෙනම model එකක්** fit කරන්න පුළුවන් – මේක බොහෝ විට වඩා ඵලදායීයි.
- Cluster කරද්දී කණ්ඩායම් හැදෙන්නේ **මොන ලක්ෂණ** නිසාද කියලා අපි කලින් දන්නේ නැහැ. Clustering න් පස්සේ **ගැඹුරින් හාරා බලන්න** ඕන: cluster 1 සහ cluster 2 හි observations බලලා ඒවා වෙනස් කරන ප්‍රධාන ලක්ෂණ (හෝ සංයෝජන) හොයන්න – උදා: ස්ත්‍රී/පුරුෂ භාවය, ජීවත් වන ස්ථානය, ජන විකාශන හෝ සමාජීය සාධක. සමහර විට clusters සරලව විස්තර කරන්න බැහැ.
- Clustering කියන්නේ **මූලික හෝ අතරමැදි පියවරක්**: **data pre-process කිරීම → cluster කිරීම → model කිරීම**. අපි කවදාවත් clustering න් නවතින්නේ නැහැ.
- අපි කාණ්ඩ කරන්නේ **observations** (පේළි), **variables නෙවෙයි**.
- Label එකක් (response variable එකක්) නැති නිසා clustering කියන්නේ **unsupervised learning**.

2. පළමු practical එක: iris data මත k-means

ශිෂ්‍යයෝ clustering ක්‍රමයක් විදිහට **k-means** කිව්වා. "k-means" කියන්නේ data **කලින් තීරණය කළ කණ්ඩායම් ගණනකට, k** බෙදීම. k = 2, 3, 4, 5 … සඳහා run කරලා, within/between sum of squares, scree වගේ (elbow) plot එකක් හෝ silhouette plot එකක් බලලා clusters ගණන තීරණය කරන්න පුළුවන්.

ක්‍රියාකාරකම: iris data set එක load කරලා k-means clustering යොදලා clusters කීයක් පේනවද කියන්න. (Function එක දන්නේ නැත්නම් help page එක `?kmeans` open කරලා අවශ්‍ය arguments බලන්න.)

2.1 Data load කරලා බැලීම

```
# STA 3402 – Clustering: K-means clustering
data(iris)
dim(iris)        # obs 150, variables 5
head(iris)
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa

2.2 Quantitative variables විතරක් ඇයි?

PCA (dimension reduction) වගේම මූලික clustering එකත් **quantitative data විතරයි** භාවිතා කරන්නේ. ඇයි? Clustering පදනම් වෙන්නේ **observations අතර දුර** මත: ළඟ points එකම cluster එකට දානවා, ඈත points අතහරිනවා. දුරක් ගණනය කරන්න (උදා: Euclidean දුර – "x වෙනස සහ y වෙනස අරන්, වර්ග කරලා, එකතු කරලා, වර්ගමූලය ගන්න", පාසලේ ඉගෙනගත්තා වගේ) **අංක** ඕන.

පුද්ගලයා	වයස	ස්ත්‍රී/පුරුෂ
1	37	පිරිමි
2	48	ගැහැණු
3	90	පිරිමි

අවුරුදු 37 පිරිමියෙක් අවුරුදු 48 ගැහැණියකගෙන් කොච්චර දුරද? වයස් වෙනස මනින්න පුළුවන්, නමුත් **"පිරිමි" සහ "ගැහැණු" අතර දුර මනින්න බැහැ**. ඒ නිසා මේ මූලික clustering එකට quantitative variables විතරක් තියාගන්නවා. (Categorical data වලට වෙනත් ක්‍රම ඕන – උදා: පසුගිය lecture එකේ වගේ MCA/FAMD coordinates.)

2.3 Data copy කරලා numeric columns තියාගැනීම

```
# make a copy of the dataset
iris_data = iris
# extract only the quantitative variables
str(iris_data)                 # Species කියන්නේ Factor එකක් (5 වන variable එක)
iris_quants = iris_data[-5]    # 5 වන column එක හැර ඔක්කොම
```

- හැමවිටම **copy** එකක් මත වැඩ කරන්න. මුල් එක වෙනස් කළොත් ඉතිරි analysis එකට (හෝ එකම data එකෙන් එන exam ප්‍රශ්නයකට) ඔක්කොම variables නැති වෙන්න පුළුවන්.
- `str()` එකෙන් Species කියන්නේ **factor** එකක් බව පෙන්නනවා. මෙතන data set එක හරියට නිර්වචනය කරලා තියෙනවා; නැත්නම් categorical variables හොයාගන්න variable විස්තර කියවන්න ඕන.
- `iris_data[-5]` column 5 ඉවත් කරනවා (data frame එකක් කෙලින්ම columns වලින් index කරන්න පුළුවන්).

2.4 k = 2, 3, 4 සඳහා k-means run කිරීම

```
# k-means
kmeans(iris_quants, 2)   # 77.6 %
kmeans(iris_quants, 3)   # 88.4 %
table(iris_data$Species)
kmeans(iris_quants, 4)   # 89.5 %
```

Distance matrix එකක් ගණනය කරන්න **ඕන නැහැ** – `kmeans()` ඔක්කොම කරනවා. අපි දෙන්නේ numeric data සහ clusters ගණන විතරයි (k = 1 තේරුමක් නැහැ: එහෙනම් cluster කරන්නේ ඇයි?).

k = 2 සඳහා output එක:

```
K-means clustering with 2 clusters of sizes 53, 97
Cluster means:
  Sepal.Length Sepal.Width Petal.Length Petal.Width
1     5.005660    3.369811     1.560377    0.290566
2     6.301031    2.886598     4.958763    1.695876
Clustering vector:  1 1 1 1 1 ... 2 2 2 ...
Within cluster sum of squares by cluster:
[1]  28.55208 123.79588
 (between_SS / total_SS =  77.6 %)
Available components:
"cluster" "centers" "totss" "withinss" "tot.withinss" "betweenss" "size" "iter" "ifault"
```

Output කොටස	**තේරුම**
sizes	හැම cluster එකකම observations ගණන
Cluster means (`$centers`)	හැම cluster එකකම හැම variable එකකම mean එක – cluster "මධ්‍යය"
Clustering vector (`$cluster`)	හැම observation එකක්ම අයිති cluster එක
Within cluster SS (`$withinss`)	හැම cluster එකක්ම කොච්චර පැතිරිලාද (කුඩා = තද cluster එකක්)
between_SS / total_SS	Clusters අතර වෙන්වීමෙන් පැහැදිලි කරන මුළු විචලනයේ % (100% ට ළඟ = හොඳයි)

k	**between_SS / total_SS (lecture)**	**`nstart = 25` සමඟ**
2	77.6%	77.6%
3	88.4%	88.4%
4	89.5%	91.6%

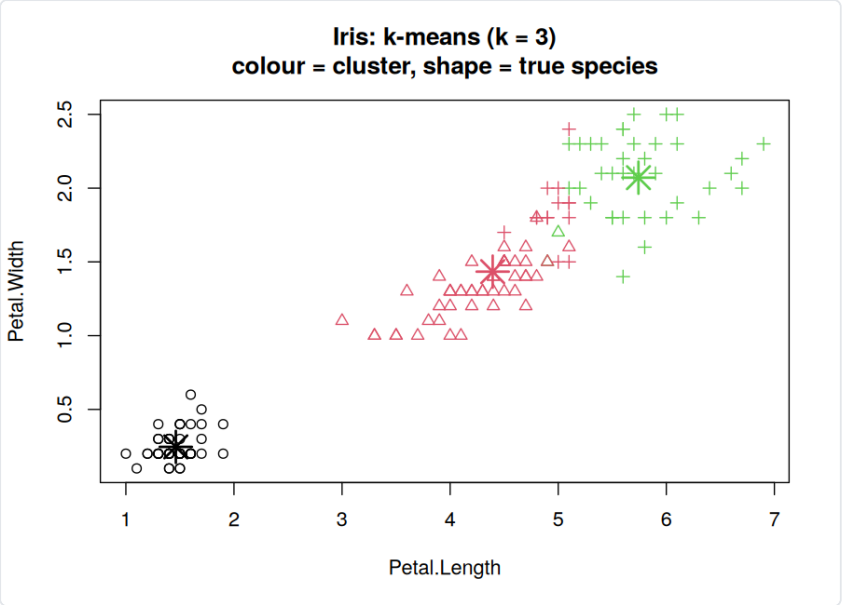
> **ඔයාගේ අංක වෙනස් වෙන්න පුළුවන් ඇයි?**
> k-means පටන් ගන්නේ **අහඹු (random)** centres වලින්, ඒ නිසා හැම run එකක්ම වෙනස් (සමහර විට නරක) විසඳුමකින් ඉවර වෙන්න පුළුවන්. උදාහරණයක් විදිහට `kmeans(iris_quants, 3)` එක run එකකින් ලැබුණේ 79.0% විතරයි. නැවත ලබාගත හැකි ප්‍රතිඵල සඳහා `set.seed(123)` ද, `nstart = 25` (අහඹු ආරම්භ 25ක් උත්සාහ කරලා හොඳම එක තියාගන්න) ද භාවිතා කරන්න: `kmeans(iris_quants, 3, nstart = 25)`.

2.5 ඇත්ත species එක්ක සසඳීම

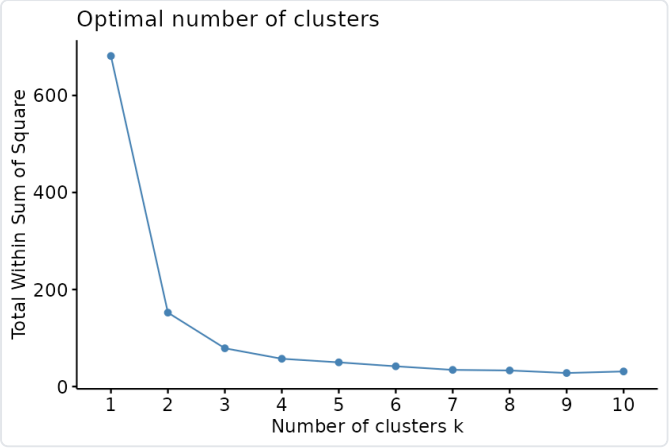
```
table(iris_data$Species)     # setosa 50, versicolor 50, virginica 50
```

Data set එකේ ඇත්තටම **මල් species තුනක්** තියෙනවා: setosa, versicolor සහ virginica. Clustering ට කලින් Species ඉවත් කළ නිසා **label එකක් තිබුණේ නැහැ** – ඒ නිසා මේක unsupervised learning. Species භාවිතා කරන්නේ පස්සේ, ප්‍රතිඵලය check කරන්න විතරයි:

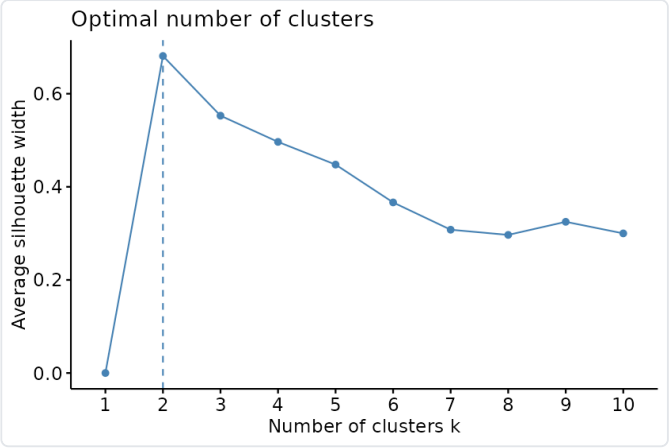
Cluster (k = 3)	setosa	versicolor	virginica
1	50	0	0
2	0	48	14
3	0	2	36



Setosa සම්පූර්ණයෙන්ම වෙන් වෙලා; versicolor සහ virginica ටිකක් මිශ්‍ර වෙනවා



Elbow plot: `fviz_nbclust(iris_quants, kmeans, method = "wss")`



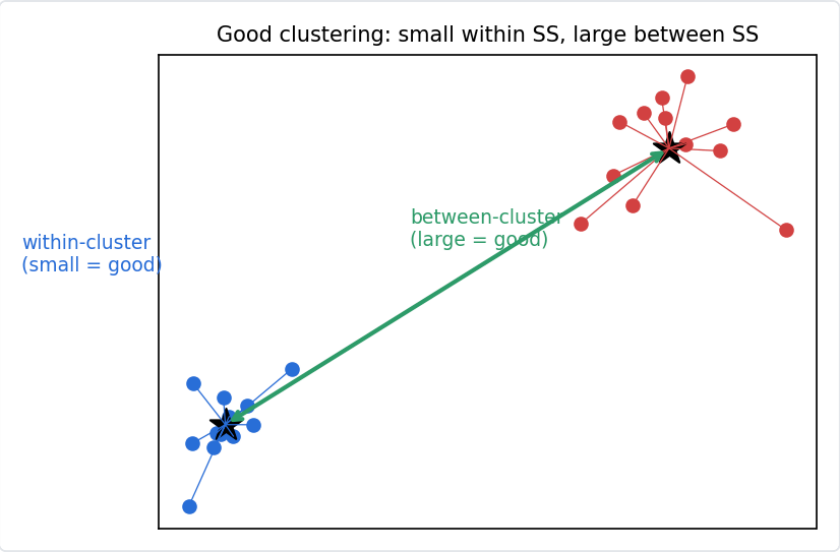
Silhouette plot: `method = "silhouette"`

k-means – වාසිය සහ අවාසිය

වාසිය: ඉතා සරලයි, වේගවත්.

අවාසිය: algorithm එක run කරන්න **කලින්** clusters ගණන **k** තීරණය කරන්න ඕන, මොකද k කියන්නේ inputs වලින් එකක්.

3. Within සහ between sum of squares

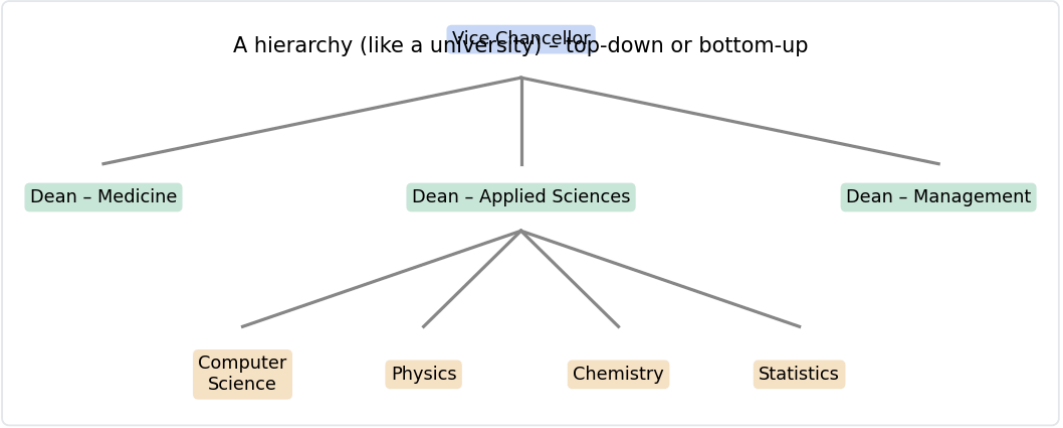


- Data හොඳින් cluster වෙලා නම් හැම cluster එකක් **ඇතුළේ** දුර කුඩායි → **within-cluster sum of squares කුඩායි**.
- Clusters එකිනෙකට ඈතින් → **between-cluster sum of squares විශාලයි**.
- Total SS = Within SS + Between SS. ඒ නිසා අපි **Between SS / Total SS** බලනවා. වැඩි වෙන්න වෙන්න හොඳයි (100% නම් පරිපූර්ණයි). Clusters 2ක් සමඟ iris සඳහා 77.6% – හොඳයි.
- Clusters වැඩි කළාම මේ අනුපාතය හැමවිටම ටිකක් වැඩි වෙනවා, ඒ නිසා වාසිය කුඩා වෙන තැන (**elbow**) සහ clusters වලට තේරුමක් ඇති තැන k තෝරන්න.

4. සමස්ත චිත්‍රය: clustering වර්ග

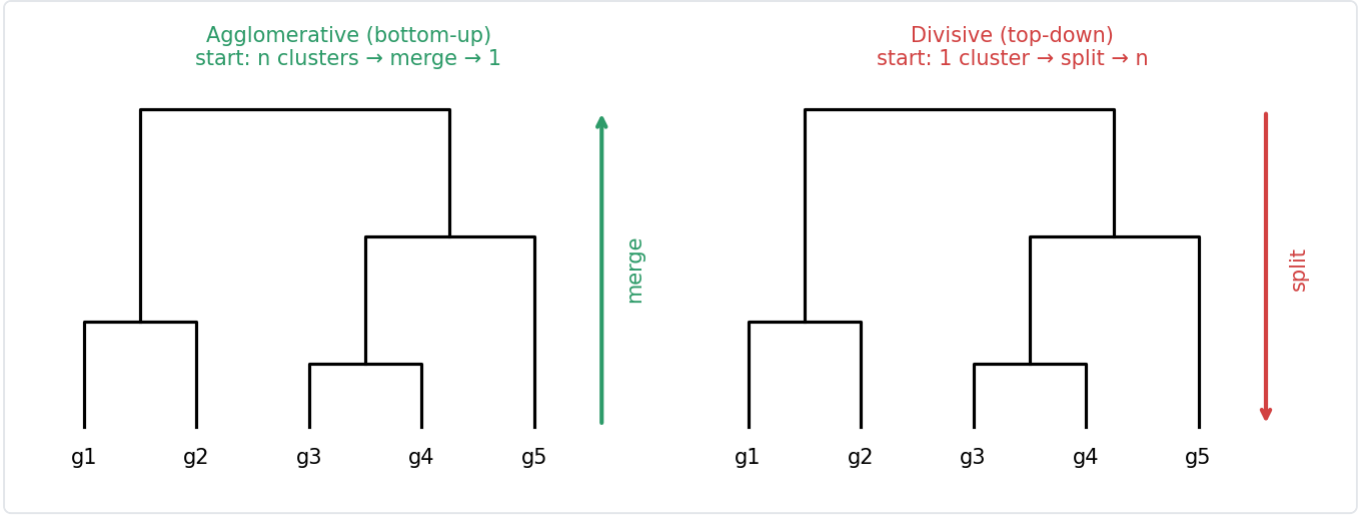
Clustering වල **label තොරතුරු නැහැ**, ඒ නිසා ඒක **unsupervised** machine learning. ප්‍රධාන වර්ග දෙකක් තියෙනවා:

1. **k-means** (partitioning) clustering
2. **Hierarchical** clustering



ධුරාවලියක් (hierarchy) කියන්නේ විශ්වවිද්‍යාලයක් වගේ: ඉහළින්ම උපකුලපති, ඊට පස්සේ පීඨාධිපතිවරුන් සහිත පීඨ (වෛද්‍ය, ව්‍යවහාරික විද්‍යා, කළමනාකරණ...), ඊට පස්සේ අංශ (පරිගණක

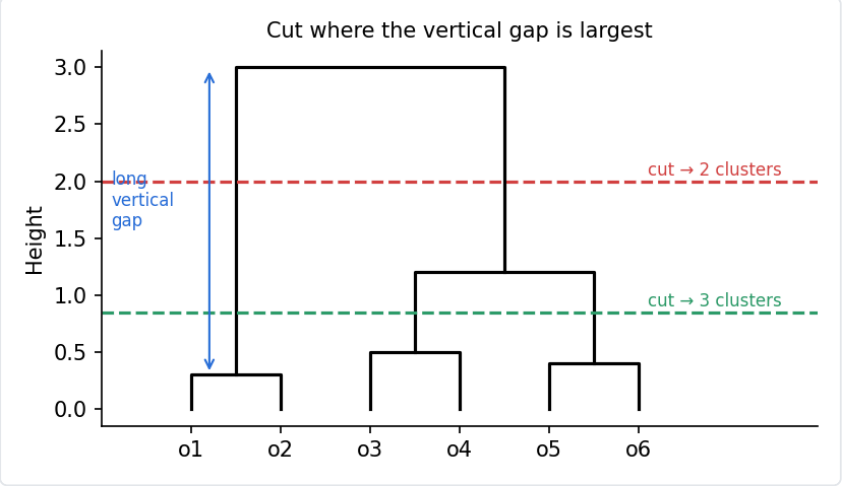
විද්‍යාව, භෞතික විද්‍යාව, රසායන විද්‍යාව, සංඛ්‍යාන විද්‍යාව...). ඒක **ඉහළ සිට පහළට** හෝ **පහළ සිට ඉහළට** කියවන්න පුළුවන්.



	Divisive clustering	**Agglomerative clustering**
දිශාව	**ඉහළ සිට පහළට (top-down)**	**පහළ සිට ඉහළට (bottom-up)**
ආරම්භය	ඔක්කොම observations සහිත එක cluster එකක් (root)	හැම observation එකක්ම තමන්ගේම cluster එකක් (clusters n, leaves)
ක්‍රියාවලිය	වඩාත්ම වෙනස් (විෂම) cluster එක යම් ලක්ෂණ අනුව බෙදනවා	වඩාත්ම සමාන clusters එකතු කරනවා
අවසානය	හැම observation එකක්ම තමන්ගේම cluster එකක	එක විශාල cluster එකක්
R	`diana()` (cluster package)	`hclust()` (stats package)

Hierarchical clustering එකක ගස් රූපසටහනට **dendrogram** කියනවා. මේ module එක ප්‍රධාන වශයෙන් agglomerative clustering ගැන, නමුත් divisive clustering ත් උගන්වනවා.

Dendrogram එකකින් clusters කීයක්ද?



Dendrogram එක හරහා **සිරස් දුර (පරතරය) සාපේක්ෂව විශාල** තැනකින් **තිරස් රේඛාවක්** අඳින්න (ඉතා කුඩා තැනකින් නෙවෙයි). ඊට පස්සේ **ඒ රේඛාව සිරස් අතු කීයක් කපනවද ගණන් කරන්න**. ඒ අංකය තමයි යෝජිත clusters ගණන.

සටහන: මේ වෙනකම් අපි cluster කරන්නේ quantitative data විතරයි (පසුගිය වතාවේ PCA වගේ). Categorical variables වලට වෙනත් ක්‍රම ඕන; lecturer ඒ ගැන ආයෙත් කතා කරනවා.

5. කිරි උදාහරණය (slides)

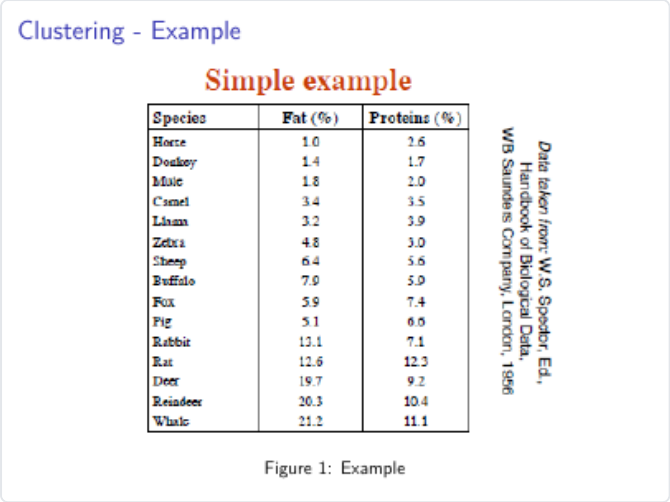


Figure 1: ක්ෂීරපායී කිරි වල සංයුතිය

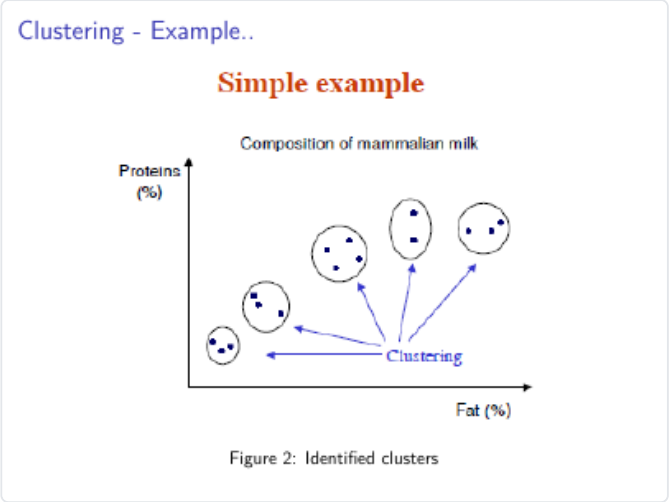


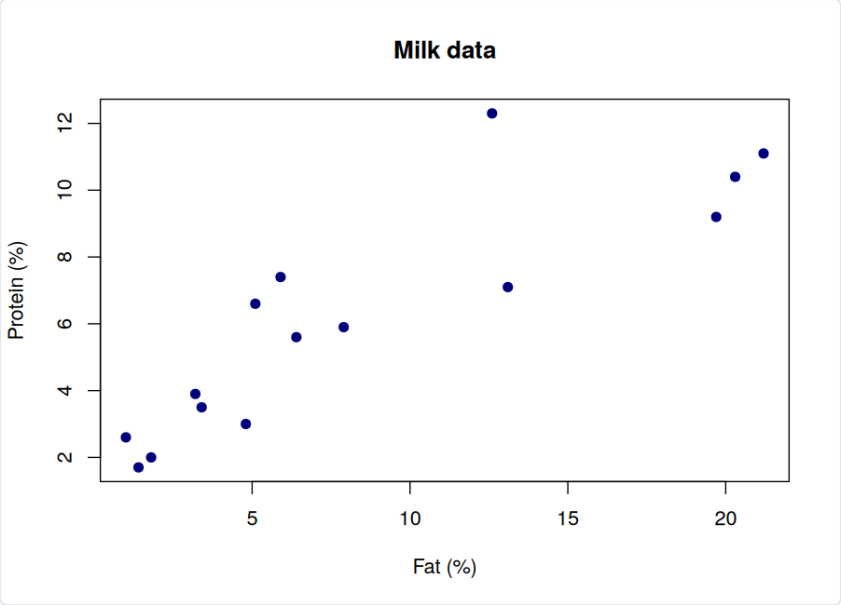
Figure 2: හඳුනාගත් clusters

ක්ෂීරපායීන් (කිරි බොන සතුන්): අශ්වයා, බූරුවා, mule, ඔටුවා, llama, zebra, බැටළුවා, මී හරකා, නරියා, හාවා, මුවා, reindeer, තල්මසා... හැම සත්ත්වයෙකුගේම කිරි වල **මේද %** සහ **ප්‍රෝටීන් %** දන්නවා – variables දෙකක්. Plot එකෙන් **clusters පහක් පමණ** පේනවා.

Lecturer ගේ tip එක: table එකක **screenshot** එකක් අරන්, AI tool එකකින් CSV file එකකට extract කරගෙන R එකට load කරන්න පුළුවන්. නැත්නම් කෙලින්ම data type කරන්න:

```
milk_data = data.frame(
  Fat     = c(1,1.4,1.8,3.4,3.2,4.8,6.4,7.9,5.9,5.1,13.1,12.6,19.7,20.3,21.2),
  Protein = c(2.6,1.7,2.0,3.5,3.9,3,5.6,5.9,7.4,6.6,7.1,12.3,9.2,10.4,11.1))

plot(y = milk_data$Protein, x = milk_data$Fat)
```



> **Error: "figure margins too large"**
>
> `Error in plot.new(): figure margins too large` කියන්නේ **RStudio හි Plots pane එක ඉතා කුඩායි**. Pane එකේ මායිම ඇදලා ලොකු කරන්න (පහළ panel එක දිගු කරන්න), ඊට පස්සේ `plot()` ආයෙත් run කරන්න. (වෙනත් විසඳුම්: device එක reset කරන්න `dev.off()`, හෝ `par(mar = c(4,4,1,1))` එකෙන් කුඩා margins.)

ජනප්‍රිය clustering algorithms (slides): k-means, hierarchical clustering, self-organizing maps (SOM) සහ fuzzy clustering. අපි k-means සහ hierarchical ගැන අවධානය යොමු කරනවා.

6. දුර (Distance)

ඕනෑම cluster analysis එකක **දුර** පිළිබඳ සංකල්පය ඉතා වැදගත්. (මිහින්තලේ campus එකේ සිට ඔයාගේ ගම – ගාල්ල, මාතර, මහනුවර, කොළඹ, කුරුණෑගල, ත්‍රිකුණාමලය... – දක්වා දුර ගැන හිතන්න.)

6.1 සංඛ්‍යා දෙකක් අතර දුර

$$d(a, b) = |a - b| = \sqrt{(a - b)^2}$$

ඒක **සමමිතිකයි (symmetric)**: $d(a, b) = |a - b| = \sqrt{(a - b)^2} = |b - a| = d(b, a)$.

උදාහරණ 1: $a = 1$, $b = 3 \rightarrow d(1, 3) = \sqrt{(1 - 3)^2} = \sqrt{4} = 2$, සහ $d(3, 1) = \sqrt{(3 - 1)^2} = \sqrt{4} = 2$. එකමයි.

6.2 Vectors අතර Euclidean දුර

රෝගීන් කිහිප දෙනෙකුගේ gene expression අගයන් තියෙනකොට $a = (a_1, \ldots, a_n)$ සහ $b = (b_1, \ldots, b_n)$ වගේ **vectors** අතර දුර ඕන. වඩාත් පොදු එක **Euclidean දුර = වර්ග කළ වෙනස්කම් වල එකතුවේ වර්ගමූලය**:

$$d(a, b) = \sqrt{\left(\Sigma \, (a_i - b_i)^2 \right)}$$

උදාහරණ 2: $a = (1, 1)$, $b = (4, 5)$:

$$d(a, b) = \sqrt{((1-4)^2 + (1-5)^2)} = \sqrt{(9+16)} = \sqrt{25} = 5$$

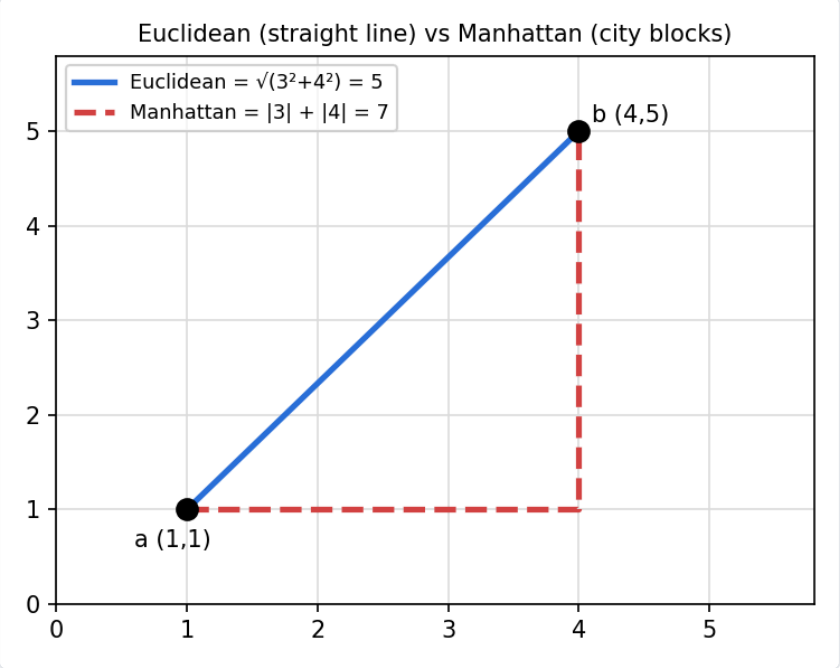
මේක පයිතගරස්: තිරස් පැත්ත 3, සිරස් පැත්ත 4, ඒ නිසා කර්ණය $\sqrt{(3^2 + 4^2)} = 5$.

උදාහරණ 3 (R වල):

```
a <- c(1,1)
b <- c(4,5)
sqrt(sum((a-b)^2))        # 5

# ඔයාගේම function එකක් – ඕනෑම vectors දෙකකට වැඩ කරනවා
euclid <- function(a, b) {
  return(sqrt(sum((a - b)^2)))
}
euclid(c(1,1), c(4,5))    # 5
```

6.3 Manhattan දුර



- **Euclidean** = සරල රේඛාව ("කපුටා පියාඹන විදිහට"), උදා: මිහින්තලේ නගරයේ සිට කෙලින්ම ව්‍යවහාරික විද්‍යා පීඨයට.
- **Manhattan** = ගොඩනැගිලි (පාරවල් දෙකක් අතර blocks) උඩින් යන්න බැහැ, ඒ නිසා **blocks වටේ** යනවා: එක පාරක් දිගේ, ඊට පස්සේ තවත් පාරක් දිගේ. New York හි Manhattan දූපතේ (එක පැත්තකින් Hudson ගඟ, වටේට Brooklyn සහ New Jersey) වීදි grid එකක් වගේ, මිනිස්සු "blocks දෙකක් පහළට ගිහින් වමට හැරිලා තව blocks දෙකක් යන්න" වගේ මඟ පෙන්වනවා.
- **සූත්‍රය:** $d(a, b) = \Sigma |a_i - b_i|$. a = (1,1), b = (4,5) සඳහා: |3| + |4| = 7.
- R වල: `dist(x, method = "manhattan")`. `?dist` හි වෙනත් options: "euclidean" (default), "maximum", "canberra", "binary", "minkowski".

7. Hierarchical clustering: පියවර සහ linkage

වර්ග දෙක (slides)

- **Agglomerative:** හැම observation එකක්ම තමන්ගේම cluster එකක් (**leaf**) විදිහට පටන් ගන්නවා; එක විශාල cluster එකක් (**root**) ඉතිරි වෙනකම් වඩාත්ම සමාන clusters පියවරෙන් පියවර එකතු කරනවා. පහළ සිට ඉහළට.
- **Divisive:** root එකෙන් පටන් ගන්නවා (ඔක්කොම objects එක cluster එකක); හැම observation එකක්ම තනි වෙනකම් වඩාත්ම විෂම clusters පියවරෙන් පියවර බෙදනවා. ඉහළ සිට පහළට.

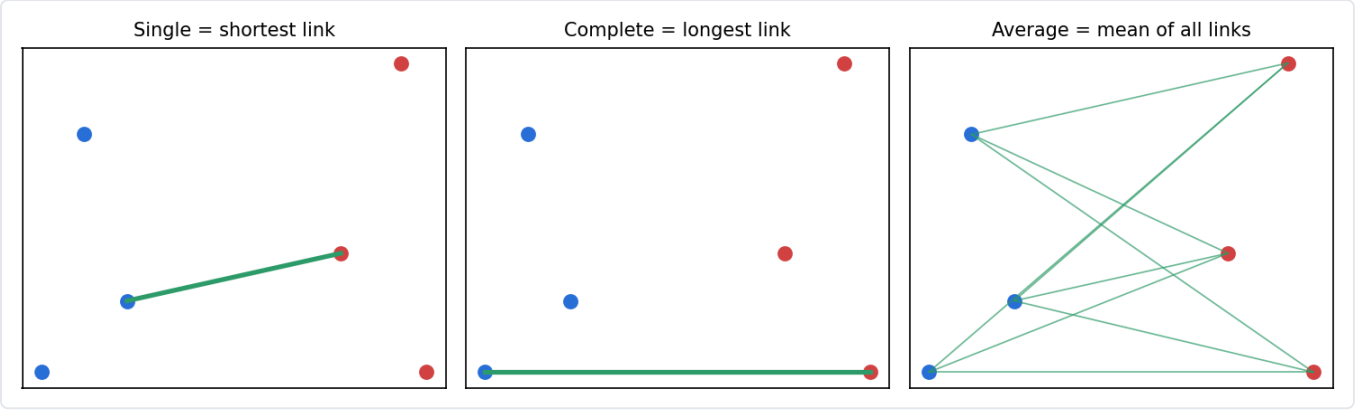
ප්‍රතිඵලය **dendrogram** කියන ගස් ආකාරයේ නිරූපණයක්.

7.1 Agglomerative hierarchical clustering හි පියවර

1. **Data සූදානම් කිරීම** (numeric, missing values හසුරුවලා, සාමාන්‍යයෙන් **scale** කරලා).
2. හැම objects යුගලයක් අතරම **දුර (dissimilarity) ගණනය කිරීම**.
3. දුර තොරතුරු මත පදනම්ව objects hierarchical cluster ගසකට කාණ්ඩ කරන්න **linkage function** එකක් භාවිතා කිරීම. ළඟ ඇති objects/clusters එකට සම්බන්ධ කරනවා.
4. ගස clusters වලට **කපන්නේ** කොතැනින්ද කියලා තීරණය කිරීම.

ඉතින් **අමුද්‍රව්‍ය දෙකක්** තියෙනවා: **දුර** සහ **linkage ක්‍රමය**. උදාහරණය: A–B දුර = 2, A–C දුර = 4. අපි සම්බන්ධ කරන්නේ මොනවද, සහ *කණ්ඩායමක* සිට තවත් කණ්ඩායමකට දුර මනින්නේ කොහොමද? ඒක තීරණය කරන්නේ linkage එක.

7.2 Linkage ක්‍රම – clusters දෙකක් අතර දුර මනින විදිහ



Linkage	Clusters I සහ J අතර දුර	සටහන්
Single	I හි සාමාජිකයෙක් සහ J හි සාමාජිකයෙක් අතර **කුඩාම** දුර (ළඟම අසල්වාසීන්)	දිගු "දම්වැල් (chains)" හදන්න පුළුවන්
Complete (maximum)	සාමාජිකයන් අතර **විශාලතම** දුර (අඩුම සමාන සාමාජිකයන් දෙදෙනා); එකතු කිරීමෙන් මේ උපරිම දුර අවම කරනවා	තද, ගෝලාකාර clusters දෙනවා. `hclust()` හි default එක
Average	I සහ J හි සාමාජිකයන් අතර ඔක්කොම දුරවල **සාමාන්‍යය**	Single සහ complete අතර
Ward	**Within-cluster sum of squares** අඩුවෙන්ම වැඩි වෙන යුගලය එකතු කරනවා	Variance අවම කරන ක්‍රමයක්, k-means objective එකට සමානයි නමුත් agglomerative විදිහට කරනවා. R වල: `"ward.D2"`

Single linkage සූත්‍රය (slides): $d(I, J) = \min \{ d(x_i, x_j) : x_i \in I \text{ හි සහ } x_j \in J \text{ හි } \}$. ඒ කියන්නේ clusters දෙකක් අතර දුර කියන්නේ ඒවායේ **ළඟම අසල්වාසීන්** අතර දුර.

Single linkage algorithm එක: data points ගණනට සමාන clusters ගණනකින් පටන් ගන්නවා → ළඟම දෙක හොයලා එකතු කරනවා → ඊළඟ ළඟම clusters දෙක හොයලා එකතු කරනවා → ඔක්කොම points එක cluster එකක් වෙනකම් දිගටම කරනවා.

> මේ පාඨමාලාවේදී ක්‍රම පිටුපස ඇති ගණිතමය න්‍යායෙන් ටිකක් ලැබෙනවා – Harvard හෝ Stanford පාඨමාලා වල තරම් matrix algebra නැහැ ("තනුක කළ මාත්‍රාවක්"), නමුත් අත්‍යවශ්‍ය දේවල් තියෙනවා. විෂය බොහෝ දුරට ප්‍රායෝගිකයි.

8. Genes පහක් සමඟ single linkage උදාහරණය

පුද්ගලයන් දෙදෙනෙකුගෙන් gene expressions පහක්: g1 = (1, 1), g2 = (1, 1.1), g3 = (3, 2), g4 = (3, 2.3), g5 = (5, 5). හැම gene එකකම අගයන් p1 සහ p2 කියන අක්ෂ දෙකක coordinates.

8.1 Data ඇතුළත් කිරීම – පේළි observations විය යුතුයි

```
# distance
?dist
df <- matrix(c(1,1,1,1.1,3,2,3,2.3,5,5),
             nrow = 2, byrow = FALSE)
df          # 2 x 5: පේළි = පුද්ගලයන්, columns = genes
t(df)       # 5 x 2: පේළි = genes (observations), columns = p1, p2
d = dist(t(df))
d
```

- ශිෂ්‍යයෙක් අංක 10 එක vector එකක් විදිහට ඇතුළත් කරලා matrix එකක් හැදුවා. `nrow = 2, byrow = FALSE` සමඟ R column එකෙන් column එක පුරවනවා, 2 × 5 matrix එකක් (පේළි වල පුද්ගලයන්) ලැබෙනවා.
- `dist()` දුර ගණනය කරන්නේ **පේළි අතර** (`?dist` බලන්න). අපිට ඕන **genes** අතර දුර, ඒ නිසා genes පේළි විය යුතුයි.

- සංඛ්‍යානයේ/data science වල data set එකක සාමාන්‍යයෙන් **observations පේළි වල**, **variables columns වල**. ඒ නිසා අපි `t(df)` එකෙන් **transpose** කරනවා – data වෙනස් කරන්නේ නැහැ, transpose කළ version එක `dist()` ට දෙනවා විතරයි.
- `dist()` හි default ක්‍රමය **Euclidean**, ඒ නිසා ඒක කියන්න ඕන නැහැ.

	g1	g2	g3	g4
g2	0.10			
g3	2.24	2.19		
g4	2.39	2.33	0.30	
g5	5.66	5.59	3.61	3.36

කියවන විදිහ: g4 පේළියේ, g3 column එකේ අගය d(g4, g3) = 0.30. විකර්ණය (gene එකක් තමන් එක්ක) 0 වෙනවා, ඉහළ භාගය පහළ භාගයට සමානයි, ඒ නිසා R පෙන්නන්නේ පහළ ත්‍රිකෝණය විතරයි.

> R file එකේ `g(4,3) = 0.300000` පේළිය matrix එක කියවන විදිහ ගැන **සටහනක්** විතරයි – R code එකක් නෙවෙයි. ඒකේ ඉස්සරහට `#` එකක් දාන්න, නැත්නම් R error එකක් දෙනවා.

Slides වල row සහ column නම් දීලා එකම දේ එක පියවරකින් කරනවා:

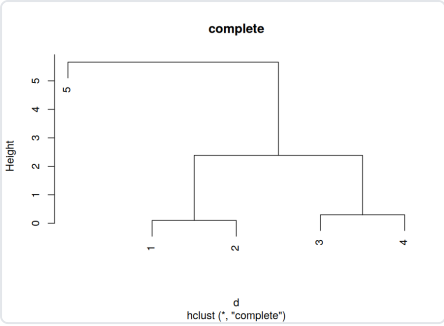
```
names <- list(c("g1","g2","g3","g4","g5"), c("p1","p2"))
sl.clus.dat <- matrix(c(1,1,1,1.1,3,2,3,2.3,5,5), ncol = 2,
                      byrow = TRUE, dimnames = names)
print(dist(sl.clus.dat, method = "euclidean"), digits = 3)
plot(sl.clus.dat, type = "n", xlim = c(0,6), ylim = c(0,6))
text(sl.clus.dat, labels = row.names(sl.clus.dat))
```

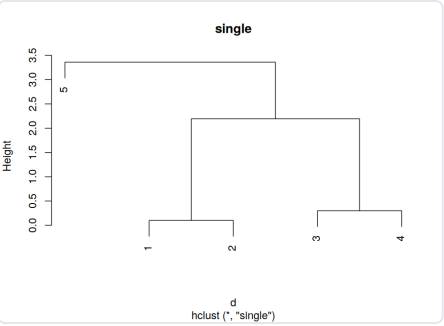
8.2 hclust() සමඟ hierarchical clustering

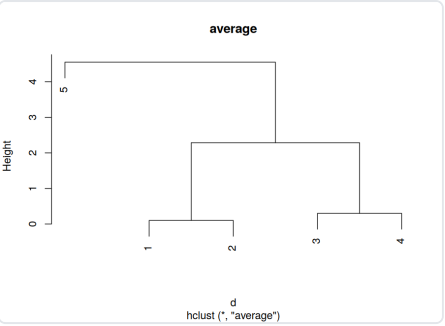
```
# hierarchical
?hclust                                          # stats package, default method = "complete"
hclust_out = hclust(d, method = "complete")      # data නෙවෙයි, DISTANCES දෙන්න
hclust_out
summary(hclust_out)                              # එච්චර ප්‍රයෝජනවත් නැහැ
plot(hclust_out)                                 # dendrogram එක
```

- `hclust()` තියෙන්නේ **stats** package එකේ – install කරන්න ඕන නැහැ.
- Input: **distance object** එක `d` (data set එක නෙවෙයි) සහ **linkage** එක (`method`). Default එක `"complete"`; ලියන එක අත්‍යවශ්‍ය නැහැ, නමුත් පැහැදිලියි.
- Print කළාම පෙන්නනවා: Cluster method: complete, Distance: euclidean, Number of objects: 5.
- `summary()` කොටස් (merge, height, order…) ලැයිස්තුගත කරනවා විතරයි, කියවන්න අමාරුයි. **Plot** එක (dendrogram) ගොඩක් ප්‍රයෝජනවත්.

```
# how many clusters? why?
hclust_out2 = hclust(d, method = "single");   plot(hclust_out2)
hclust_out3 = hclust(d, method = "average");  plot(hclust_out3)
```







එකතු වීම	Single	Complete	Average
g1 + g2	0.10	0.10	0.10
g3 + g4	0.30	0.30	0.30
{g1,g2} + {g3,g4}	min = 2.19	max = 2.39	(2.24+2.19+2.39+2.33)/4 = 2.29
+ g5	min = 3.36	max = 5.66	(5.66+5.59+3.61+3.36)/4 = 4.55

Points 5ක් විතරක් තියෙන නිසා linkages තුනෙන්ම **එකම පිළිවෙළ** ලැබෙනවා; වෙනස් වෙන්නේ උස විතරයි. වෙනත් data sets වලදී ඒවා ගොඩක් වෙනස් වෙන්න පුළුවන්.

> **Clusters කීයද, ඇයි?**
> විශාලතම සිරස් පරතරය තියෙන්නේ {g1,g2}+{g3,g4} එකතු වීමට ඉහළින් (g5 එකතු වීම දක්වා). එතනින් තිරස් රේඛාවක් ඇන්දොත් **අතු 2ක්** කපනවා → clusters 2: {g1,g2,g3,g4} සහ {g5}. 0.30 ට ටිකක් ඉහළින් රේඛාවක් ඇන්දොත් **අතු 3ක්** කපනවා → {g1,g2}, {g3,g4}, {g5}. උත්තර දෙකම පිළිගන්න පුළුවන් – මේක යම් තරමකට ආත්මීයයි; scatter plot එකෙන් කණ්ඩායම් 3ක් පැහැදිලිව පේනවා.

8.3 Single linkage – ක්‍රියා කරන විදිහ (පියවරෙන් පියවර)

1. මුලින් හැම point එකක්ම වෙනම cluster එකක්.
2. ළඟම දෙක g1 සහ g2 (d = 0.10). ඒවා එකතු කරනවා: I = {g1, g2}. ගසේ මේක 0.10 උසේ තිරස් රේඛාව. g3, g4, g5 තවමත් වෙන වෙනම.
3. ඊළඟ කුඩාම දුර d(g3, g4) = 0.30 → J = {g3, g4}, 0.30 උසේ රේඛාව. දැන් clusters තුනක්: I, J සහ K = {g5}.
4. I–J දුර = d(g1,g3)=2.24, d(g1,g4)=2.39, d(g2,g3)=2.19, d(g2,g4)=2.33 න් කුඩාම = **2.19**. J–K දුර = min(3.61, 3.36) = 3.36. I–K දුර = min(5.66, 5.59) = 5.59. කුඩාම I–J, ඒ නිසා I සහ J 2.19 දී එකතු වෙනවා.
5. අන්තිමට {g1, g2, g3, g4} සහ g5 අතර දුර = d(g4, g5) = **3.36** → අන්තිම එකතු වීම.

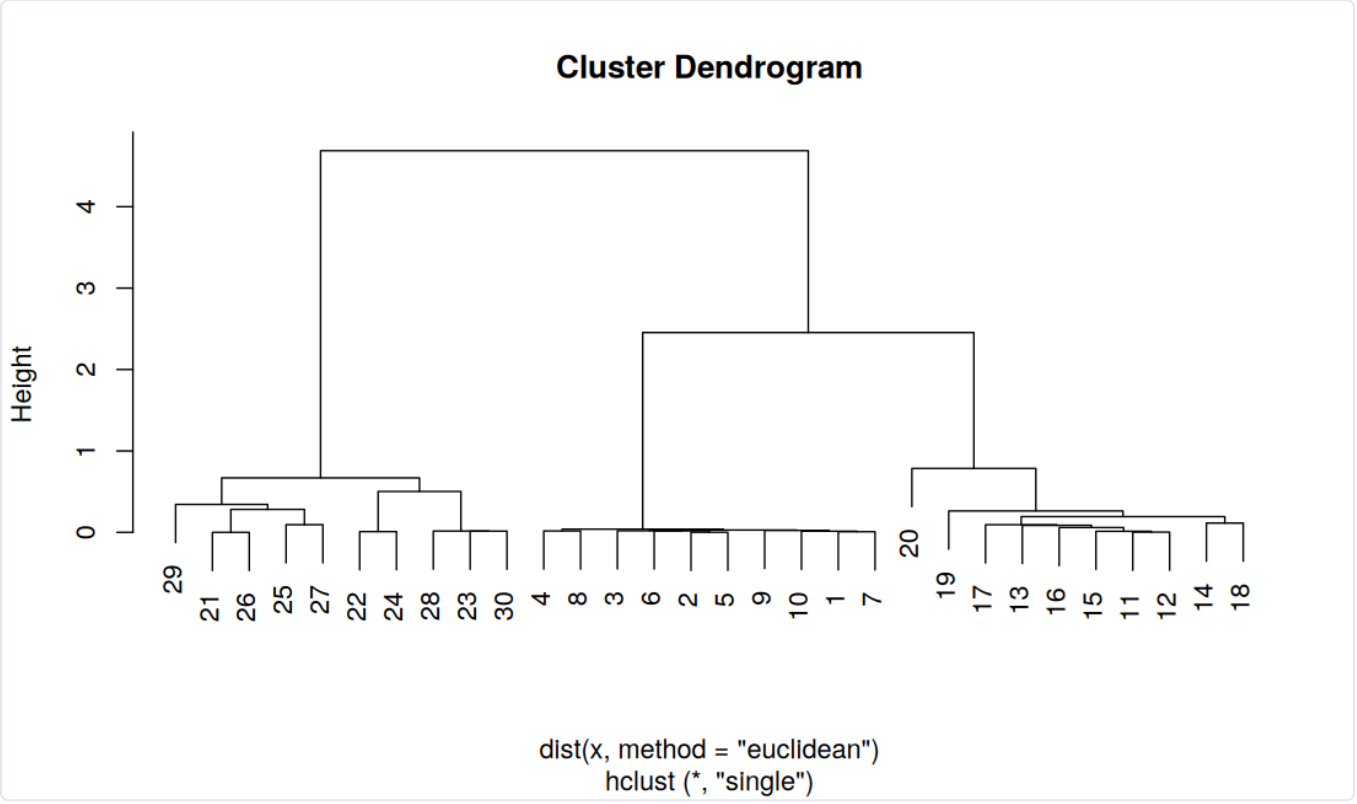
සාමාන්‍ය R dendrogram එකක plot එකට පහළින් **දුර ක්‍රමය** සහ **linkage ක්‍රමය** පෙන්නනවා (උදා: "dist(..., method = "euclidean")", "hclust (*, "single")").

9. උදාහරණ 2: normal populations තුනකින් data

මේ උදාහරණයෙන් **data හැදෙන ක්‍රියාවලිය (data generation process)** cluster ගසට සම්බන්ධ කරනවා. Data අහඹු නිසා එකම අංක ලබාගන්න seed එක සකසනවා.

```
set.seed(100)
x = c(rnorm(10, 0, 0.1),     # points 10, mean 0,  sd 0.1
      rnorm(10, 3, 0.5),     # points 10, mean 3,  sd 0.5
      rnorm(10, 10, 1.0))    # points 10, mean 10, sd 1
plot(hclust(dist(x, method = "euclidean"), method = "single"))
# OR මුලින් store කරන්න
sl.out = hclust(dist(x, method = "euclidean"), method = "single")
plot(sl.out)
```

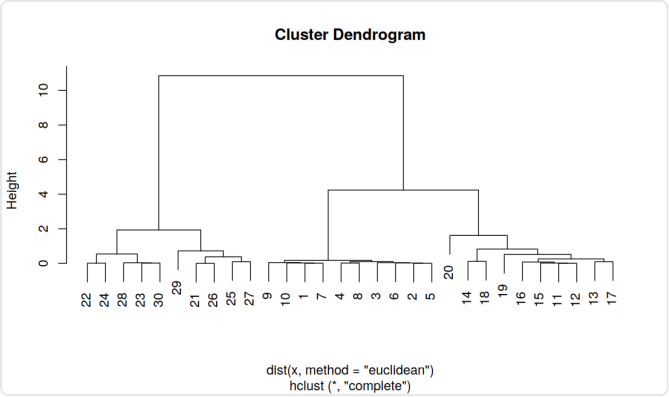
- `rnorm(n, mean, sd)` normal distribution එකකින් අහඹු අංක n ක් හදනවා. අගයන් 30 කණ්ඩායම් තුනකින් එන නිසා **clusters 3ක්** බලාපොරොත්තු වෙනවා.
- Observations 1–10 කණ්ඩායම 1 න්, 11–20 කණ්ඩායම 2 න්, 21–30 කණ්ඩායම 3 න්.



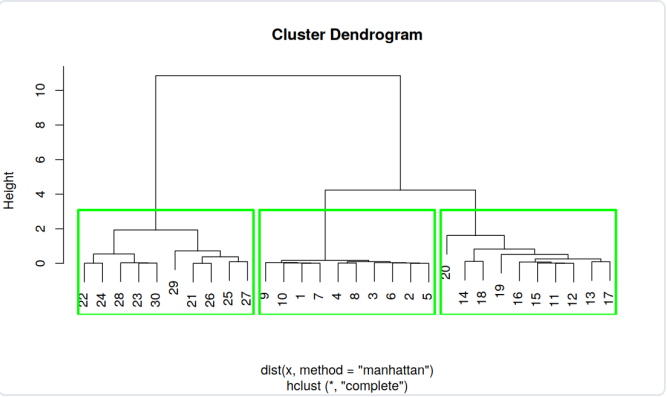
Single linkage, Euclidean දුර

විශාලතම සිරස් දුර බලලා තිරස් රේඛාවක් අඳින්න. ඉහළින් රේඛාවක් **clusters 2ක්** දෙනවා; පහළින් රේඛාවක් **clusters 3ක්** දෙනවා. මෙතන උත්තර දෙකම හරි. ප්‍රායෝගිකව ඒක **ක්ෂේත්‍රය** මත රඳා පවතිනවා: ඔයා ජීව විද්‍යාඥයෙක් හෝ ආර්ථික විද්‍යාඥයෙක් එක්ක වැඩ කරනවා නම් clusters අර්ථවත්ද කියලා ඔවුන් check කරනවා.

```
plot(hclust(dist(x, method = "euclidean"), method = "complete"))   # linkage වෙනස් කිරීම
plot(hclust(dist(x, method = "manhattan"), method = "complete"))   # දුර වෙනස් කිරීම
clm.out = hclust(dist(x, method = "manhattan"), method = "complete")
plot(clm.out)
rect.hclust(clm.out, k = 3)                       # clusters 3 වටේ කොටු
rect.hclust(clm.out, k = 3, border = "green")     # පාට වෙනස් කිරීම
```



Complete linkage – ටිකක් වෙනස්



Manhattan + complete, `rect.hclust(k = 3)`

- **Linkage** එක හෝ **දුර** වෙනස් කළාම ගස වෙනස් වෙනවා. හොඳම cluster සැකැස්ම හොයන්න විවිධ සංයෝජන උත්සාහ කරන්න.
- `rect.hclust(tree, k = 3)` k clusters වටේ සෘජුකෝණාස්‍ර අඳිනවා. `border` පාට සකසනවා.
- පන්තියේදී lecturer රේඛා ඝන කරන්න උත්සාහ කළා. `rect.hclust()` හි **lwd argument එකක් නැහැ** (arguments: tree, k, which, x, h, border, cluster). වැඩ කරන ක්‍රමයක්: `op <- par(lwd = 3); rect.hclust(clm.out, k = 3, border = "green"); par(op)`.
- Clusters 3ක් සමඟ observations 1–10, 11–20 සහ 21–30 වෙනම clusters තුනකට වැටෙනවා – data හැදුණු විදිහටමයි.

10. k-means clustering – න්‍යාය

k-means bioinformatics සහ තවත් බොහෝ ක්ෂේත්‍රවල ජනප්‍රියයි. ඒක K clusters පුරා **within-cluster sum of squares අවම කරනවා**. Data points $x_1, \ldots, x_n$ දුන්නාම, මේක අවම කරනවා:

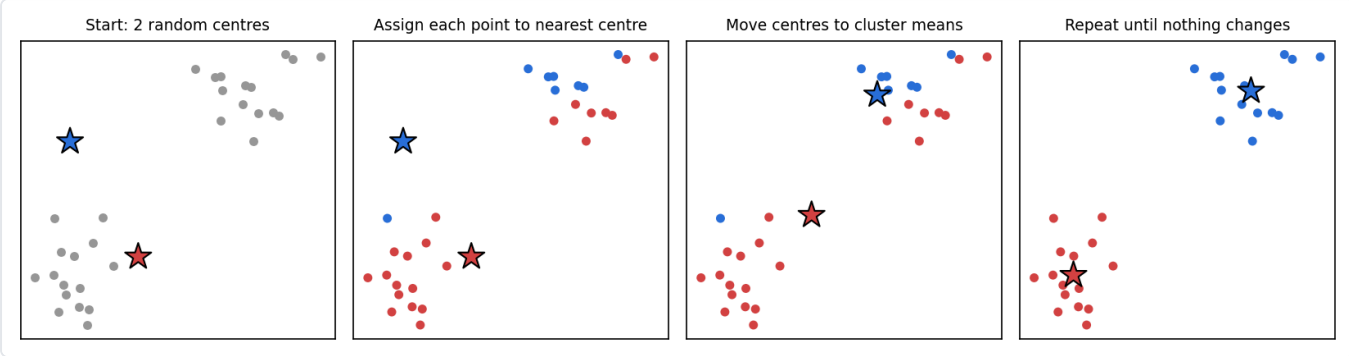
$$\sum_{k=1}^{K} \sum_{i \in I_k} d^2(x_i, a_k)$$

හැකි සියලුම මධ්‍ය points $a_1, \ldots, a_K$ පුරා.

- දුර වෙනස්කම් ධන හෝ ඍණ වෙන්න පුළුවන්, ඒ නිසා **වර්ග කළ** දුර භාවිතා කරනවා (ලකුණු නැහැ).
- Indices දෙකක්: cluster k = 1 සඳහා, cluster 1 හි හැම point එකකටම $d^2(x, a_1)$ එකතු කරන්න (උදා: points 15ම). ඊට පස්සේ cluster 2 ට එකම දේ කරන්න, ආදී වශයෙන්. ඔක්කොම එකතු කළාම **මුළු within SS** ලැබෙනවා.

Algorithm එක (Hartigan & Wong, 1979)

1. Data අහඹු ලෙස හෝ යම් heuristic එකකින් ආරම්භක clusters K කට බෙදන්න.
2. **පියවර 1:** cluster means ගණනය කරන්න.
3. **පියවර 2:** හැම point එකක්ම **ළඟම cluster mean එකට** assign කරලා අලුත් බෙදීමක් හදන්න.
4. **අභිසාරී (convergence)** වෙනකම් – කිසිම data point එකක් cluster වෙනස් නොකරන තුරු – පියවර 1 සහ 2 නැවත නැවත කරන්න.



11. උදාහරණ 3: simulate කළ data මත k-means සහ dendrograms

පුද්ගලයන් දෙදෙනෙකු සඳහා normal populations දෙකකින් gene expressions simulate කරනවා: N(0, 0.5) න් 50ක් සහ N(2, 0.5) න් 50ක්. ඒවා 50 × 2 matrices දෙකක එකක් යට අනිත් එක තියලා → data points 100ක්. ඊට පස්සේ 2-means run කරනවා.

```
set.seed(100)
data <- rbind(matrix(rnorm(100, 0, 0.5), ncol = 2),
              matrix(rnorm(100, 2, 0.5), ncol = 2))
cl <- kmeans(data, 2)
cl
```

- `rnorm(100, 0, 0.5)` → අංක 100ක්; `matrix(..., ncol = 2)` → පේළි 50 × columns 2.
- `rbind()` = **row bind**: දෙවැනි matrix එක පළමු එකට **යටින්** තියනවා → 100 × 2 matrix එකක්.
- k-means සඳහා දුර ගණනය කිරීමක් ඕන නැහැ – data සහ k විතරයි.
- (Slides වල `rnorm(50, …)` භාවිතා කරනවා, එතකොට 25 + 25 = points 50ක් සහ 81.2% ලැබෙනවා; R file එකේ 100 + 100 අංක = points 100ක්.)

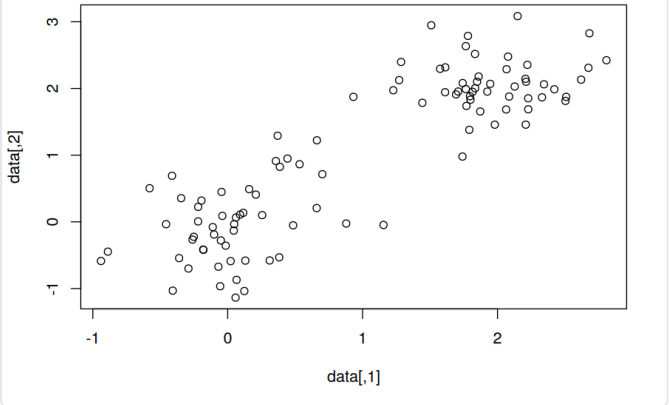
```
K-means clustering with 2 clusters of sizes 50, 50
Cluster means:
        [,1]        [,2]
1 1.95010671  2.06103413
2 0.04075447 -0.03784191
Clustering vector: 2 2 2 ... 2 1 1 1 ... 1
Within cluster sum of squares by cluster:
[1] 15.38181 25.63134
 (between_SS / total_SS =  83.1 %)
```

(පන්තියේදී එකම seed එක නැතුව lecturer ට 79.2% ලැබුණා.) Variables දෙකක් තියෙන නිසා හැම cluster mean එකකටම coordinates දෙකක් (x, y) තියෙනවා. Cluster **labels අත්තනෝමතිකයි**: observations 1–50 එක cluster එකක්, 51–100 අනිත් එක; කුමන එකට "1" කියනවද කියන එක වැදගත් නැහැ.

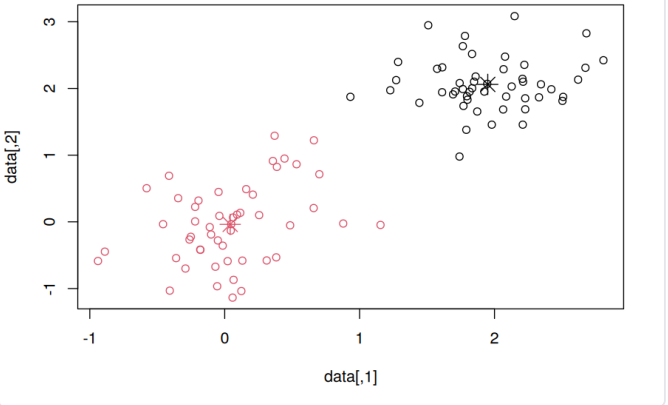
```
plot(data)                                      # raw data
plot(data, col = cl$cluster)                    # පාට = cluster label
points(cl$centers, col = 1:2, pch = 8, cex = 2)    # centres එකතු කිරීම
```

- `col = cl$cluster`: cluster labels (1, 2) පාට විදිහට භාවිතා කරනවා (1 = කළු, 2 = රතු). `c("red","black")[cl$cluster]` වගේ පාට දෙන්නත් පුළුවන්.

- `points()` දැනට තියෙන plot එකකට points එකතු කරනවා. `pch` = plotting character (8 = තරුවක්; 2 = ත්‍රිකෝණය, 19 = පිරුණු තිත උත්සාහ කරන්න). `cex` = ප්‍රමාණය.



plot(data)

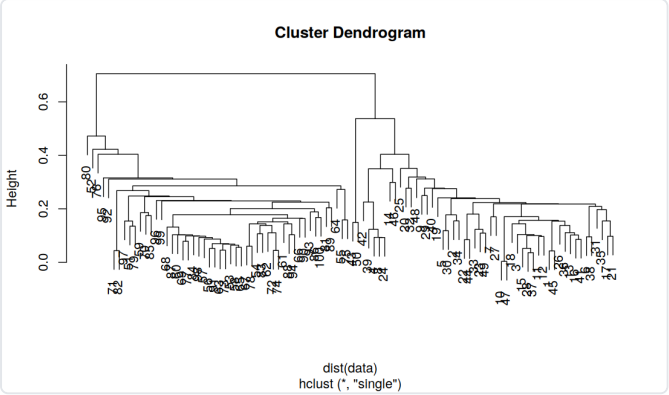


Cluster අනුව පාට කළා, තරු = centres

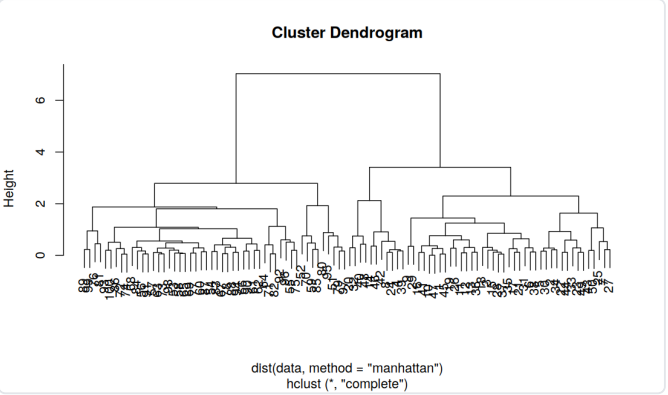
ක්‍රියාකාරකම: dendrogram එක ලබාගෙන clusters ගණන හඳුනාගන්න

```
# obtain the dendrogram
d_data = dist(data)
hclust_out4 = hclust(d_data, method = "single")
plot(hclust_out4)

d_data = dist(data, method = "manhattan")
hclust_out4 = hclust(d_data, method = "complete")
plot(hclust_out4)
```



Euclidean + single: අවුල් ("chaining")



Manhattan + complete: ගොඩක් පැහැදිලියි

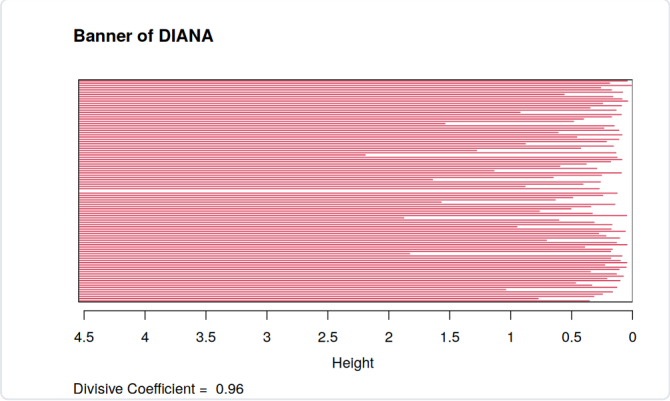
Single linkage සමඟ ගස "අමුතුයි", කියවන්න අමාරුයි. Complete linkage (සහ Manhattan දුර) ට වෙනස් කළාම ලස්සන ගසක් ලැබෙනවා. **Clusters 2ක්** පේනවා (data populations 2කින් ආපු බව දන්න නිසා ඒක තමයි නිවැරදිම), නමුත් **clusters 4ක්** පෙන්නන කැපීමකුත් පිළිගන්න පුළුවන්. මේ අත්හදා බැලීම (trial and error) තමයි cluster analysis කරන සාමාන්‍ය විදිහ.

මුලින්ම data scale කරන්න
පළමු පියවර විය යුත්තේ data **scale (standardise)** කිරීම බව lecturer සඳහන් කළා – agglomerative, divisive (සහ k-means) වලටත්. මෙතන data එකම පරිමාණයෙන් simulate කළත්, වෙනස් ඒකක ඇති ඇත්ත data සඳහා `scale()` භාවිතා කරන්න. `hclust()` හි standardise option එකක් නැහැ, ඒ නිසා ඔයාම කරන්න: pre-processing (missing values ආදිය) → standardise → cluster.

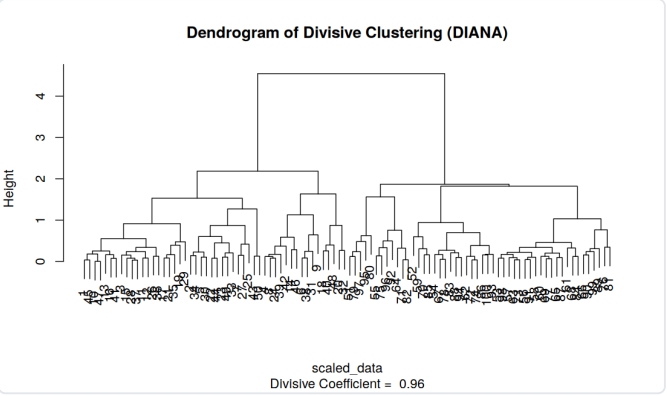
12. DIANA සමඟ divisive clustering (top-down)

```
# top down approach (divisive hierarchical clustering)
scaled_data = scale(data)
library(cluster)
hc_div_out = diana(scaled_data)
plot(hc_div_out, main = "Dendrogram of Divisive Clustering (DIANA)")
```

- **DIANA** = DIvisive ANAlysis. `diana()` function එක තියෙන්නේ **cluster** package එකේ (R එක්කම එනවා; load කරන්න විතරයි).
- (Scale කළ) data දෙන්න; වෙන කිසිවක් ඕන නැහැ.
- Output එකේ objects වල පිළිවෙළ, උස සහ **divisive coefficient** = මෙතන **0.96** (1 ට ළඟ = ශක්තිමත් clustering ව්‍යුහයක්).
- `plot()` plots දෙකක් දෙනවා; ඊළඟ එක බලන්න console එකේ **Enter** ඔබන්න: මුලින් **banner** plot එක, ඊට පස්සේ **dendrogram** එක.
- Divisive එක cluster එකකින් පටන් අරන් බෙදනවා; agglomerative clusters n කින් පටන් අරන් එකතු කරනවා.



Banner plot



DIANA dendrogram (divisive coefficient 0.96)

```
k_clusters = cutree(hc_div_out, k = 2)    # ගස clusters 2කට කපන්න
table(k_clusters)                         # 50 50
```

`cutree()` හැම observation එකකම cluster අංකය දෙනවා (k-means හි `$cluster` වගේ). ඒක `hclust()` ගස් වලටත් වැඩ කරනවා. මෙතන DIANA හි clusters 2 k-means clusters එක්ක හරියටම ගැලපෙනවා (50 / 50).

factoextra සමඟ ලස්සන plots

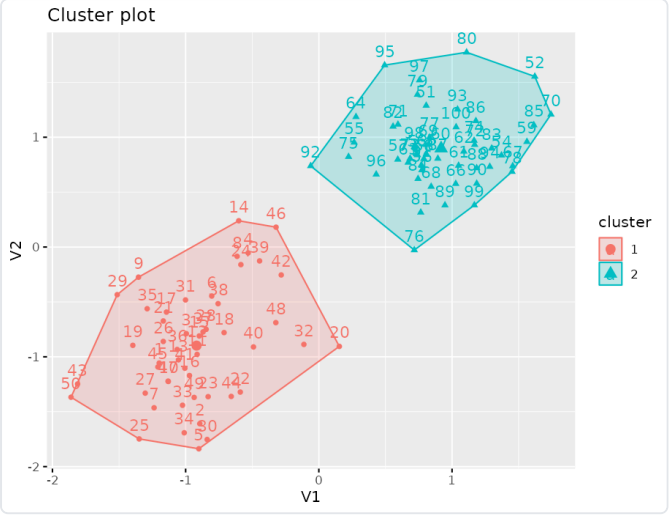
```
#install.packages("factoextra")   # එක වතාවයි
library(factoextra)
fviz_cluster(data = scaled_data, cluster = k_clusters)     # lecture code – ERROR
```

fviz_cluster() සඳහා විසඳුම

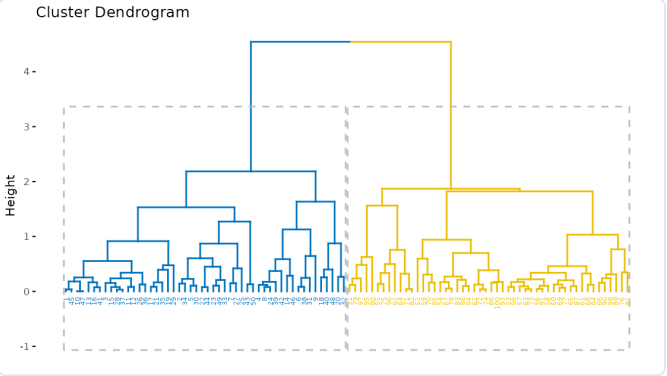
පන්තියේදී මේ පේළිය වැඩ කළේ නැහැ ("argument "object" is missing"). `fviz_cluster()` ට data සහ clusters සහිත **list** එකක් ඕන, සහ data වලට **column නම්** තිබිය යුතුයි:

```
colnames(scaled_data) <- c("V1", "V2")
fviz_cluster(list(data = scaled_data, cluster = k_clusters))
fviz_dend(hc_div_out, k = 2, rect = TRUE, cex = 0.4, palette = "jco")   # පාට කළ dendrogram
```

මේ අමතර package codes කටපාඩම් කරන්න ඕන නැහැ – හොයලා බලන්න (උදා: Datanovia website එක, එතන USArrests data සමඟ `diana(USArrests, stand = TRUE)` භාවිතා කරනවා).



fviz_cluster: clusters 2



පාට කළ clusters 2ක් සමඟ fviz_dend

13. ගෙදර වැඩ (05.01.2026 ප.ව. 6 ට කලින්)

```
# HW: Due: 05.01.2026 by 6pm
# Use the USArrests dataset
data("USArrests")
# Carry out a cluster analysis.
# How many clusters do you see from
## 1. hierarchical agglomerative clustering with euclidean distance and complete linkage
## 2. divisive clustering
# confirm your choice above by using a K-means clustering.
```

- **USArrests**: එක්සත් ජනපද ප්‍රාන්තයකට (Alabama, Alaska, …) එක පේළිය බැගින්, Murder, Assault, UrbanPop සහ Rape අනුපාත සමඟ.
- බෙදාගන්නා link එක හරහා (LMS හෝ වෙනම link එකක්) submit කරන්න – WhatsApp හෝ email හරහා **නෙවෙයි**.

- **AI tools භාවිතා කරන්න එපා.** වැඩ AI වලින් හැදුවා කියලා හඳුනාගත්තොත් lecturer **ලකුණු බිංදුව** දෙනවා. ඒ නිසා මේ සටහන් වල උත්තර නැහැ – පහත පියවර භාවිතා කරලා ඔයාගේම code සහ අර්ථකථනය ලියන්න.

ඉගෙනගත් ක්‍රම වල checklist එක (ඔක්කොම මේ සටහන් වල පෙන්වා ඇත)

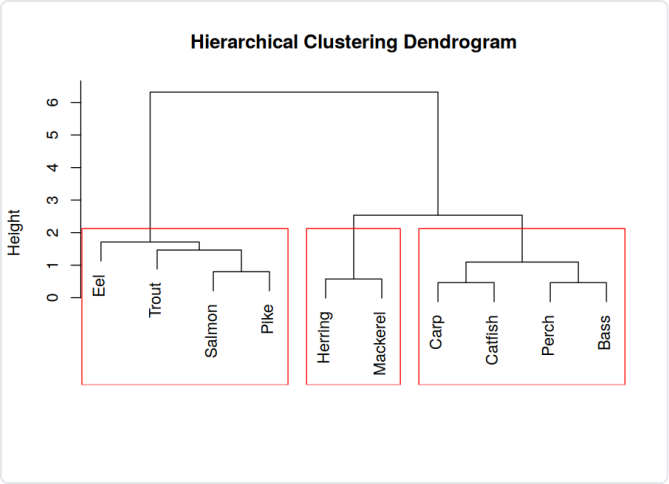
1. Data බලන්න (`head` , `str` , `summary`), missing values check කරන්න, numeric variables තියාගන්න.
2. Data **scale** කරන්න (variables ගොඩක් වෙනස් ඒකක වල).
3. Agglomerative: `dist(..., method = "euclidean")` → `hclust(..., method = "complete")` → dendrogram → සිරස් පරතරය විශාලම තැනින් කපන්න → `rect.hclust()` , `cutree()` .
4. Divisive: `diana()` → dendrogram → `cutree()` .
5. ඔයා තෝරාගත් k සමඟ k-means (`set.seed` , `nstart`), between_SS / total_SS check කරන්න, `table()` එකෙන් cluster සාමාජිකත්වය සසඳන්න.
6. ඒ clusters ගණන තෝරාගත්තේ ඇයි කියලා ඔයාගේම වචන වලින් පැහැදිලි කරන්න.

14. අමතර පුහුණුව: මාළු data සහ penguins data

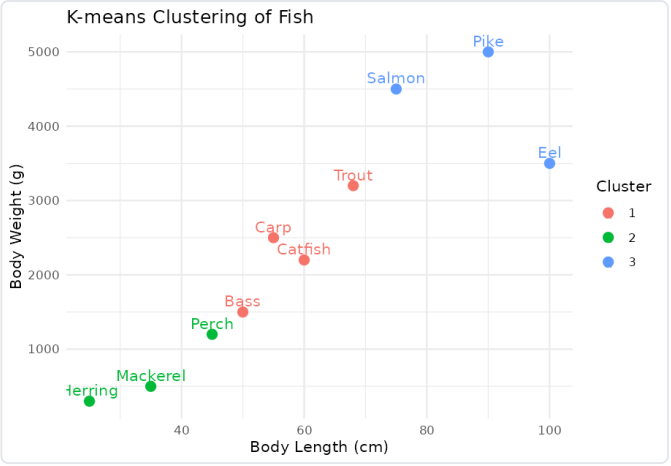
14.1 මාළු (සත්ත්ව විද්‍යා) data – hierarchical එදිරිව k-means

```
fish_data <- data.frame(
  Species =
c("Salmon","Trout","Carp","Pike","Perch","Bass","Catfish","Eel","Herring","Mackerel"),
  Body_Length_cm = c(75, 68, 55, 90, 45, 50, 60, 100, 25, 35),
  Body_Weight_g  = c(4500, 3200, 2500, 5000, 1200, 1500, 2200, 3500, 300, 500),
  Fin_Length_cm  = c(20, 18, 15, 22, 12, 14, 13, 25, 8, 10))
fish_numeric <- fish_data[, -1]          # species column එක ඉවත් කිරීම
fish_scaled  <- scale(fish_numeric)       # scale – නැත්නම් බර (grams) ආධිපත්‍යය ගන්නවා
dist_matrix <- dist(fish_scaled, method = "euclidean")
hc <- hclust(dist_matrix, method = "ward.D2")
plot(hc, labels = fish_data$Species, main = "Hierarchical Clustering Dendrogram", xlab = "",
sub = "")
rect.hclust(hc, k = 3, border = "red")
```

- මෙතන scale කිරීම අත්‍යවශ්‍යයි: බර grams දහස් ගණනින්, දිග cm දස ගණනින්. Scale නොකළොත් බර විතරක් clusters තීරණය කරයි.
- `labels = fish_data$Species` row අංක වෙනුවට මාළු නම් පෙන්නනවා; `xlab = "", sub = ""` ගසට පහළ text ඉවත් කරනවා.



Ward.D2: {Eel, Trout, Salmon, Pike}, {Herring, Mackerel}, {Carp, Catfish, Perch, Bass}



Scale කළ data මත k-means (k = 3)

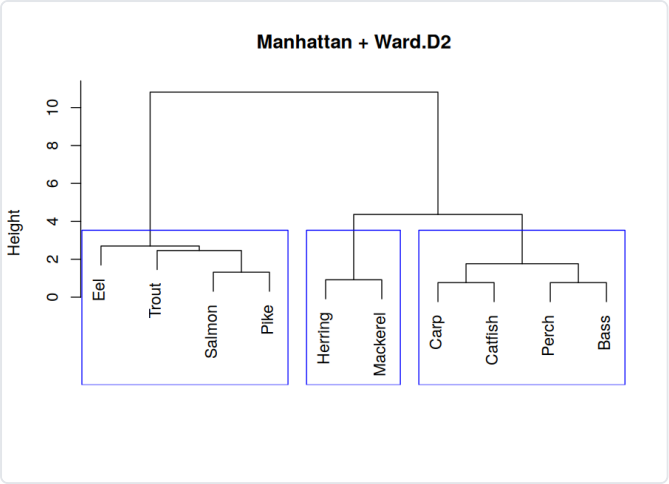
```
set.seed(123)                          # නැවත ලබාගත හැකි ප්‍රතිඵල සඳහා
kmeans_result <- kmeans(fish_scaled, centers = 3, nstart = 25)
fish_data$Cluster <- as.factor(kmeans_result$cluster)
fish_data
library(ggplot2)
ggplot(fish_data, aes(x = Body_Length_cm, y = Body_Weight_g, color = Cluster, label =
Species)) +
  geom_point(size = 3) + geom_text(vjust = -0.5) +
  labs(title = "K-means Clustering of Fish", x = "Body Length (cm)", y = "Body Weight (g)") +
  theme_minimal()
# comparing methods
table(Hierarchical = cutree(hc, k = 3), KMeans = kmeans_result$cluster)
```

ක්‍රමය	කණ්ඩායම A	කණ්ඩායම B	කණ්ඩායම C
Hierarchical (Ward)	Salmon, Trout, Pike, Eel	Carp, Perch, Bass, Catfish	Herring, Mackerel
k-means	Salmon, Pike, Eel	Trout, Carp, Bass, Catfish	Perch, Herring, Mackerel

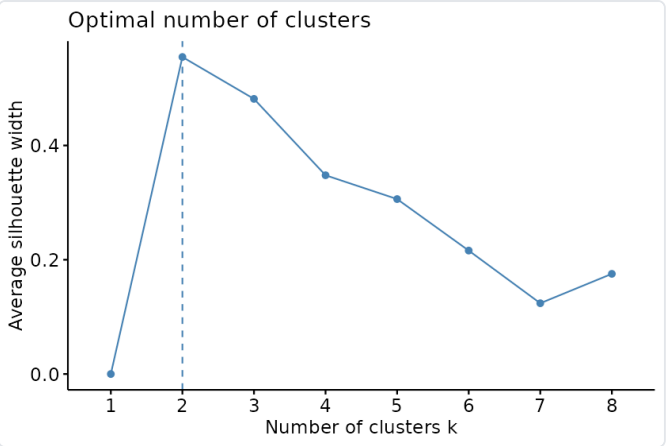
`table()` එකෙන් පෙන්නන්නේ ක්‍රම දෙක **අර්ධ වශයෙන් විතරයි එකඟ වෙන්නේ** (Trout සහ Perch මාරු වෙනවා). මාළු 10ක් විතරක් තියෙන නිසා ක්‍රමයේ කුඩා වෙනස්කම් කණ්ඩායම් වෙනස් කරනවා – ක්‍රම සසඳලා විෂය දැනුම භාවිතා කරන්න තවත් හේතුවක්.

ක්‍රියාකාරකම් (R file එකෙන්)

1. Data set එකට තව species සහ ලක්ෂණ (උදා: වලිගයේ දිග, කොරපොතු ගණන) එකතු කරන්න.
2. Hierarchical clustering එකේ වෙනත් දුරක් (උදා: `"manhattan"`) උත්සාහ කරන්න – dendrogram එක වෙනස් වෙන්නේ කොහොමද?
3. හොඳම clusters ගණන හොයන්න **NbClust** package එක භාවිතා කරන්න: `NbClust(fish_scaled, distance = "euclidean", min.nc = 2, max.nc = 6, method = "kmeans")` – ඒක indices 30ක් පමණ ගණනය කරලා බහුතරය තෝරන k යෝජනා කරනවා. (සමාන ඉක්මන් check එකක්: `fviz_nbclust(fish_scaled, kmeans, method = "silhouette")`.)
4. iris වගේ වෙනත් data set එකකට clustering යොදන්න.



ක්‍රියාකාරකම 2: Manhattan + Ward.D2

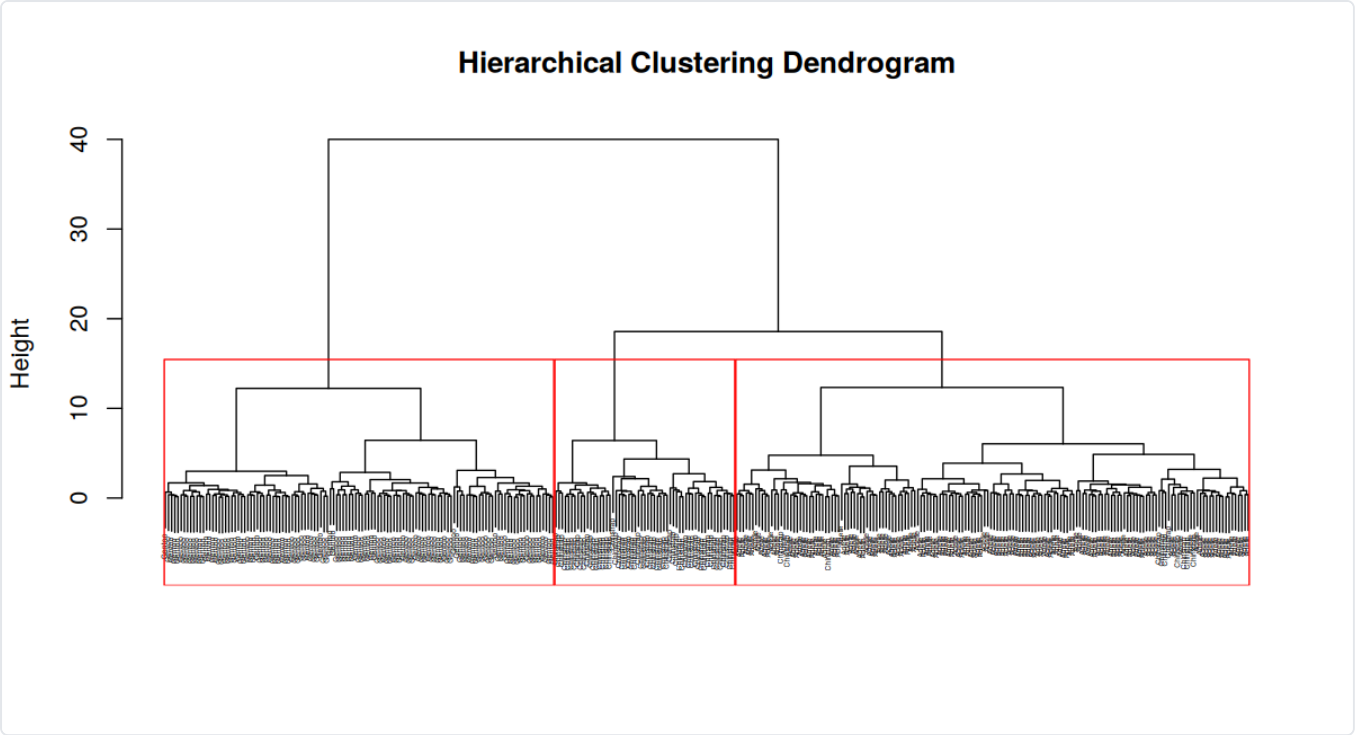


ක්‍රියාකාරකම 3 අදහස: silhouette ක්‍රමය

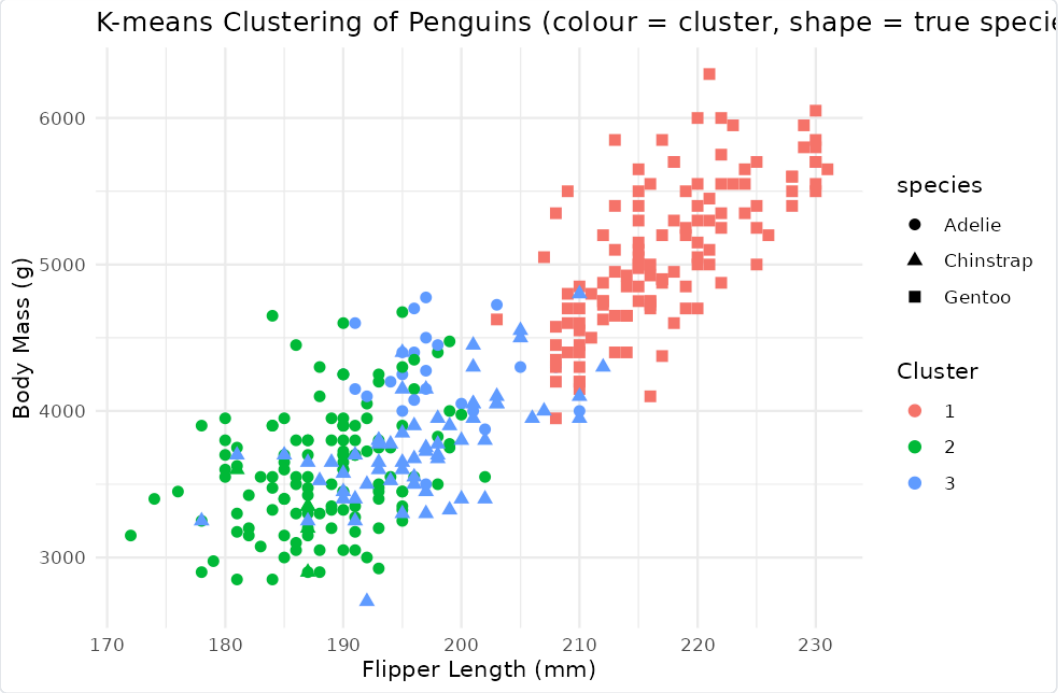
14.2 Palmer penguins

```
install.packages(c("palmerpenguins", "ggplot2", "NbClust"))   # එක වතාවයි
library(palmerpenguins); library(ggplot2); library(NbClust)
head(penguins)
penguins_clean <- na.omit(penguins[, c("species", "bill_length_mm", "bill_depth_mm",
                                       "flipper_length_mm", "body_mass_g")])
penguins_numeric <- penguins_clean[, -1]          # numeric variables විතරයි
penguins_scaled  <- scale(penguins_numeric)
dist_matrix <- dist(penguins_scaled, method = "euclidean")
hc <- hclust(dist_matrix, method = "ward.D2")
plot(hc, labels = penguins_clean$species, main = "Hierarchical Clustering Dendrogram", xlab =
"", sub = "")
rect.hclust(hc, k = 3, border = "red")
set.seed(123)
kmeans_result <- kmeans(penguins_scaled, centers = 3, nstart = 25)
penguins_clean$Cluster <- as.factor(kmeans_result$cluster)
head(penguins_clean)
```

- `na.omit()` missing values ඇති පේළි ඉවත් කරනවා (penguins 344 → 342).
- Numeric මිනුම් 4ක්, ඇත්ත species 3ක් (Adelie, Chinstrap, Gentoo) – ප්‍රතිඵලය check කරන්න විතරයි භාවිතා කරන්නේ.



Clusters 3ක් සහිත Ward.D2 dendrogram එක



k-means clusters (lecture එකේ ggplot එක geom_text එකෙන් හැම point එකකම species නම ලියනවා; මෙතන කියවීම පහසු වෙන්න species හැඩයෙන් පෙන්වා ඇත)

k-means cluster	Adelie	Chinstrap	Gentoo
1	0	0	123
2	127	5	0
3	24	63	0

Hierarchical (Ward) cluster	Adelie	Chinstrap	Gentoo
1	151	11	0
2	0	0	123
3	0	57	0

ක්‍රම දෙකෙන්ම **Gentoo සම්පූර්ණයෙන්ම** හොයාගන්නවා (විශාල ශරීරය, දිගු flippers). Adelie සහ Chinstrap ප්‍රමාණයෙන් සමාන නිසා අර්ධ වශයෙන් මිශ්‍ර වෙනවා.

15. සාරාංශය

ප්‍රධාන කරුණු

- Clustering මගින් cluster එකක සාමාජිකයන් සමාන වෙන විදිහට **observations** කාණ්ඩ කරනවා; ඒක **unsupervised**, සහ අතරමැදි පියවරක් (pre-process → cluster → model).
- මූලික clustering එකට **quantitative** data ඕන (දුර). Copy එකක් හදලා, numeric columns තියාගෙන, **scale** කරන්න.
- **k-means:** මුලින් k තෝරන්න; within SS අවම කරනවා; between_SS / total_SS check කරන්න; `set.seed` සහ `nstart` භාවිතා කරන්න; elbow/silhouette එකෙන් k තෝරන්න.
- **Hierarchical:** agglomerative (පහළ සිට ඉහළට, `hclust`) හෝ divisive (ඉහළ සිට පහළට, `diana`); ප්‍රතිඵලය = dendrogram; විශාලතම සිරස් පරතරයෙන් කපන්න.
- **දුර:** Euclidean $\sqrt{\Sigma(a_i - b_i)^2}$ (සරල රේඛාව) හෝ Manhattan $\Sigma|a_i - b_i|$ (city blocks). `dist()` වැඩ කරන්නේ **පේළි** මත.
- **Linkage:** single (min), complete (max, default), average (mean), Ward (within SS හි අවම වැඩිවීම).
- වෙනස් දුර/linkages වලින් වෙනස් ගස් ලැබෙනවා – කිහිපයක් උත්සාහ කරලා ක්ෂේත්‍ර දැනුම භාවිතා කරන්න.
- ප්‍රයෝජනවත් functions: `kmeans`, `dist`, `hclust`, `plot`, `rect.hclust`, `cutree`, `diana`, `fviz_cluster`, `fviz_dend`, `fviz_nbclust`.

ඔයාම පරීක්ෂා කරන්න

1. Clustering එකට unsupervised learning කියන්නේ ඇයි?
2. "පිරිමි" සහ "ගැහැණු" අතර දුරක් ගණනය කරන්න බැරි ඇයි?
3. between_SS / total_SS = 88.4% කියන්නේ මොකක්ද?
4. `kmeans()` run දෙකකින් වෙනස් ප්‍රතිඵල ලැබෙන්න පුළුවන් ඇයි? ඒක හදන්නේ කොහොමද?
5. (2, 3) සහ (5, 7) අතර Euclidean සහ Manhattan දුර ගණනය කරන්න. (උත්තරය: 5 සහ 7)
6. Agglomerative සහ divisive clustering අතර වෙනස පැහැදිලි කරන්න.
7. Section 8 හි single linkage සමඟ clusters {g1, g2} සහ {g3, g4} අතර දුර කීයද? Complete linkage සමඟ? (උත්තරය: 2.19 සහ 2.39)
8. Dendrogram එකකින් clusters ගණන තීරණය කරන්නේ කොහොමද?
9. `dist()` භාවිතා කරද්දී observations පේළි වල තිබිය යුත්තේ ඇයි?
10. Clustering ට කලින් මාළු data scale කළ යුත්තේ ඇයි?